/reload

> "Reload customizations (/reload [all|config|mcp|skills|agents])" (SlashCommandDef, crates/ragent-tui/src/app/state.rs).

Overview

`/reload` re-reads runtime configuration into the running TUI without a restart. With no argument it reloads everything (config, agents, MCP servers, and skills); a subcommand narrows the scope to a single subsystem. Results accumulate into one `From: /reload` report block, one status line per subsystem, with per-subsystem success (`[ok]`) or failure (`[x]`) markers.

The handler is the `reload` arm in `crates/ragent-tui/src/app/slash.rs` (approximately lines 3727-3915). As part of every reload the bash allow/deny lists and directory lists are also refreshed from config.

Syntax

```
/reload [all|config|mcp|skills|agents]
```

The first argument defaults to `all` when omitted.

Options / Subcommands

Form	Description
`/reload`	Reload everything (config + agents + MCP + skills)
`/reload all`	Same as bare `/reload`
`/reload config`	Re-read `ragent.json`, refresh provider detection, selected model, context window, thinking level, and code index flag
`/reload agents`	Rebuild the builtin and custom agent list, preserving the current selection
`/reload mcp`	Rebuild the MCP server display list from config, preserving connected status and tools
`/reload skills`	Confirm skills are re-read from disk on next use (no persistent cache)
`/reload help`, `/reload --help`, `/reload -h`	Show the help table

Examples

Reload everything

```
/reload
```

Output:

```
From: /reload
[ok] Config reloaded (ragent.json)
[ok] Agents reloaded - 3 custom agent(s) (was 3)
[ok] MCP reloaded - 2 server(s) in config (was 2)
```

```
[ok] Skills will be reloaded from disk on next use (no cache to clear)
```

Reload configuration only

```
/reload config
```

Output:

```
From: /reload
[ok] Config reloaded (ragent.json)
```

This refreshes provider detection, the selected model and its context window, the thinking level, `code_index_enabled`, and the bash and directory lists.

Reload agents only

```
/reload agents
```

Output:

```
From: /reload
[ok] Agents reloaded - 4 custom agent(s) (was 3)
```

Custom agents are rebuilt from the agent directories; when two definitions collide on a name the later one is renamed to `custom:<name>` and a diagnostic is logged. The currently selected agent is preserved when it still exists after the rebuild.

Reload MCP servers only

```
/reload mcp
```

Output:

```
From: /reload
[ok] MCP reloaded - 2 server(s) in config (was 2)
```

The display list is rebuilt from the `mcp` section of `Config::load()`. Each server keeps its previous connected status and tool list when it existed before the reload; new entries start disabled.

Reload skills only

```
/reload skills
```

Output:

```
From: /reload
[ok] Skills will be reloaded from disk on next use (no cache to clear)
```

Show help

```
/reload help
```

Output: the help table listing the `all|config|mcp|skills|agents` forms.

Output

- All output accumulates under a single `From: /reload` header; each subsystem contributes one `[ok] ...` (success) or `[x] ... reload failed: <error>` (failure) line.
- Config reload failure emits `[x] Config reload failed: <error>` and logs `reload config failed: <error>`.
- An unknown subcommand appends `Unknown subcommand '<name>'. Usage: /reload [all|config|mcp|skills|agents]` to the report.
- The status line is set to `reload`.
- Unconditionally after any `/reload` invocation, the bash lists (`ragent_agent::bash_lists::load_from_config()`) and directory lists (`ragent_agent::dir_lists::load_from_config()`) are refreshed from config.

Related

- `/config show` - inspect what will be reloaded
- `/config save` - back up the config before editing it
- `/mcp` - view the MCP server list after `/reload mcp`
- `/skills` - view the skill registry